Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

# UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2023

#RANKED 52 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED GRADE 'A' BY NAAC

PERFECT SCORE OF 150/150 AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS



UPES UNIVERSITY OF TOMORROW

# Elements of AIML

## Unit 01 – Lecture 02: Intelligent Agents & Learning Agents

Tofik Ali

School of Computer Science, UPES Dehradun

February 28, 2026

Topic focus: **Agents** as the core abstraction for AI systems

Repository: `https://github.com/tali7c/Elements-of-AIML`

Sources: Rich and Knight, *Artificial Intelligence*, McGraw Hill, 2017.;Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 1–2 (Introduction, Intelligent Agents).;Poole and Mackworth, *Artificial Intelligence: Foundations of Computational Agents* (2e), selected sections.

# Quick Links

Agents PEAS Environments Agent Types Learning Agent

# Agenda

Agents

PEAS

Environment Types

Agent Types

Learning Agent

# Learning Outcomes

By the end of this lecture, you should be able to:

- Define an **agent** and an **environment**.

# Learning Outcomes

By the end of this lecture, you should be able to:

- Define an **agent** and an **environment**.
- Create a **PEAS** description for a real task.

# Learning Outcomes

By the end of this lecture, you should be able to:

- Define an **agent** and an **environment**.
- Create a **PEAS** description for a real task.
- Classify environments (observable, deterministic, episodic, etc.).

# Learning Outcomes

By the end of this lecture, you should be able to:

- Define an **agent** and an **environment**.
- Create a **PEAS** description for a real task.
- Classify environments (observable, deterministic, episodic, etc.).
- Compare agent types (reflex, goal-based, utility-based, learning agent).

# Abbreviations & Notation

- **PEAS**: Performance measure, Environment, Actuators, Sensors

# Abbreviations & Notation

- **PEAS**: Performance measure, Environment, Actuators, Sensors
- **Percept**: what the agent observes (input)

# Abbreviations & Notation

- **PEAS**: Performance measure, Environment, Actuators, Sensors
- **Percept**: what the agent observes (input)
- **Action**: what the agent does (output)

# Abbreviations & Notation

- **PEAS**: Performance measure, Environment, Actuators, Sensors
- **Percept**: what the agent observes (input)
- **Action**: what the agent does (output)
- **State** (informal): a description of the world situation relevant to decisions

# What is an Intelligent Agent?

## Agent

An **agent** perceives its environment through **sensors** and acts upon it through **actuators**.

- Input: percepts (what the agent observes)

# What is an Intelligent Agent?

## Agent

An **agent** perceives its environment through **sensors** and acts upon it through **actuators**.

- Input: percepts (what the agent observes)
- Output: actions (what the agent does)

# What is an Intelligent Agent?

## Agent

An **agent** perceives its environment through **sensors** and acts upon it through **actuators**.

- Input: percepts (what the agent observes)
- Output: actions (what the agent does)
- Goal: maximize a performance measure / achieve objectives

# Agent Function vs Agent Program

- **Agent function**: ideal mapping from percept history $\rightarrow$ action.

# Agent Function vs Agent Program

- **Agent function**: ideal mapping from percept history $\rightarrow$ action.
- **Agent program**: real implementation running on a physical machine with limits.

# Agent Function vs Agent Program

- **Agent function**: ideal mapping from percept history $\rightarrow$ action.
- **Agent program**: real implementation running on a physical machine with limits.
- Design challenge: approximate the desired function under constraints (time, compute, sensors).

# Performance Measure vs Reward vs Utility (Intuition)

- **Performance measure**: external definition of success (what we truly want).

Good engineering: align reward/utility with performance measure to avoid unintended behavior.

# Performance Measure vs Reward vs Utility (Intuition)

- **Performance measure**: external definition of success (what we truly want).
- **Reward** (common in RL): feedback signal given during interaction.

Good engineering: align reward/utility with performance measure to avoid unintended behavior.

# Performance Measure vs Reward vs Utility (Intuition)

- **Performance measure**: external definition of success (what we truly want).
- **Reward** (common in RL): feedback signal given during interaction.
- **Utility**: internal numeric score used for trade-offs (choose best action).

Good engineering: align reward/utility with performance measure to avoid unintended behavior.

# PEAS: How to Specify a Task

PEAS = **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

## Example: Spam Filter

- P: minimize spam in inbox; avoid blocking important emails
- E: incoming emails + users + mail server policies
- A: move to spam, block sender, flag, allow
- S: email content, sender metadata, user feedback

# PEAS Example: Vacuum Cleaner Agent

- **P**: cleanliness, energy use, time, collisions

# PEAS Example: Vacuum Cleaner Agent

- **P**: cleanliness, energy use, time, collisions
- **E**: rooms, dirt distribution, obstacles, people/pets

# PEAS Example: Vacuum Cleaner Agent

- **P**: cleanliness, energy use, time, collisions
- **E**: rooms, dirt distribution, obstacles, people/pets
- **A**: move, turn, suck, dock/charge

# PEAS Example: Vacuum Cleaner Agent

- **P**: cleanliness, energy use, time, collisions
- **E**: rooms, dirt distribution, obstacles, people/pets
- **A**: move, turn, suck, dock/charge
- **S**: camera/LiDAR, bump sensors, dirt sensor, battery level

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable
- **Deterministic** vs stochastic

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable
- **Deterministic** vs stochastic
- **Episodic** vs sequential

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable
- **Deterministic** vs stochastic
- **Episodic** vs sequential
- **Static** vs dynamic

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable
- **Deterministic** vs stochastic
- **Episodic** vs sequential
- **Static** vs dynamic
- **Discrete** vs continuous

# Environment Properties (Why they matter)

The same algorithm does not work equally well in all environments.

- **Observable** vs partially observable
- **Deterministic** vs stochastic
- **Episodic** vs sequential
- **Static** vs dynamic
- **Discrete** vs continuous
- **Single-agent** vs multi-agent

# Environment Examples (Quick)

- **Chess**: fully observable, deterministic, sequential, static, discrete, multi-agent.

# Environment Examples (Quick)

- **Chess**: fully observable, deterministic, sequential, static, discrete, multi-agent.
- **Driving**: partially observable, stochastic, sequential, dynamic, continuous, multi-agent.

# Environment Examples (Quick)

- **Chess**: fully observable, deterministic, sequential, static, discrete, multi-agent.
- **Driving**: partially observable, stochastic, sequential, dynamic, continuous, multi-agent.
- **Email spam filtering**: partially observable (limited signals), stochastic, episodic-ish, discrete actions.

# Rationality (Intuition)

## Rational agent

Chooses actions that maximize expected performance given its percepts and knowledge.

- Rational $\neq$ omniscient (cannot know everything).

# Rationality (Intuition)

## Rational agent

Chooses actions that maximize expected performance given its percepts and knowledge.

- Rational $\neq$ omniscient (cannot know everything).
- Rational $\neq$ perfect (bounded by time, compute, sensors).

# Rationality (Intuition)

## Rational agent

Chooses actions that maximize expected performance given its percepts and knowledge.

- Rational $\neq$ omniscient (cannot know everything).
- Rational $\neq$ perfect (bounded by time, compute, sensors).
- We care about **expected** outcomes under uncertainty.

# Agent Types (High Level)

- **Simple reflex**: action depends only on current percept.

# Agent Types (High Level)

- **Simple reflex**: action depends only on current percept.
- **Model-based reflex**: maintains internal state about the world.

# Agent Types (High Level)

- **Simple reflex**: action depends only on current percept.
- **Model-based reflex**: maintains internal state about the world.
- **Goal-based**: chooses actions to reach goals.

# Agent Types (High Level)

- **Simple reflex**: action depends only on current percept.
- **Model-based reflex**: maintains internal state about the world.
- **Goal-based**: chooses actions to reach goals.
- **Utility-based**: chooses actions to maximize a utility function.

# Agent Types (High Level)

- **Simple reflex**: action depends only on current percept.
- **Model-based reflex**: maintains internal state about the world.
- **Goal-based**: chooses actions to reach goals.
- **Utility-based**: chooses actions to maximize a utility function.
- **Learning agent**: improves with experience.

# Agent Types (With 1-Line Examples)

- Simple reflex: IF dirt THEN suck ELSE move.

# Agent Types (With 1-Line Examples)

- Simple reflex: IF dirt THEN suck ELSE move.
- Model-based: remember last seen obstacle; plan around it.

# Agent Types (With 1-Line Examples)

- Simple reflex: IF dirt THEN suck ELSE move.
- Model-based: remember last seen obstacle; plan around it.
- Goal-based: choose actions to reach destination on a map.

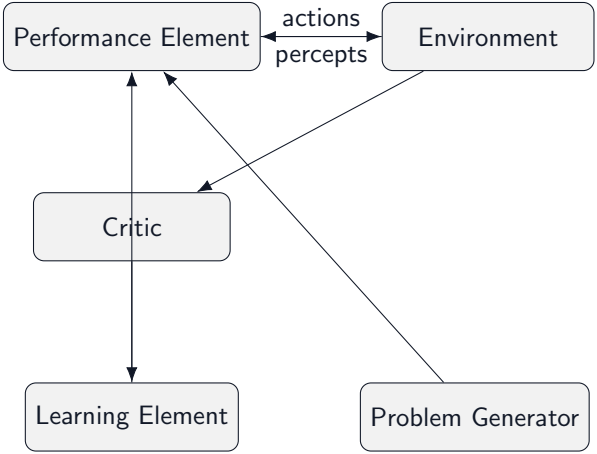
# Agent Types (With 1-Line Examples)

- Simple reflex: IF dirt THEN suck ELSE move.
- Model-based: remember last seen obstacle; plan around it.
- Goal-based: choose actions to reach destination on a map.
- Utility-based: choose fastest route that also minimizes toll cost.

# Agent Types (With 1-Line Examples)

- Simple reflex: IF dirt THEN suck ELSE move.
- Model-based: remember last seen obstacle; plan around it.
- Goal-based: choose actions to reach destination on a map.
- Utility-based: choose fastest route that also minimizes toll cost.
- Learning: improve recommendations based on user clicks/feedback.

# Learning Agent Architecture




- **Critic** evaluates behavior; **learning element** updates the agent

# Exploration vs Exploitation (Preview)

- **Exploitation**: use what you already know to get good performance now.

# Exploration vs Exploitation (Preview)

- **Exploitation**: use what you already know to get good performance now.
- **Exploration**: try new actions to learn potentially better behavior.

# Exploration vs Exploitation (Preview)

- **Exploitation**: use what you already know to get good performance now.
- **Exploration**: try new actions to learn potentially better behavior.
- Problem generator supports exploration to avoid getting stuck with suboptimal behavior.

# Summary

- Agents are the core abstraction: perceive → decide → act.
- PEAS helps specify tasks clearly.
- Environment properties guide algorithm choices.
- Learning agents improve with experience and feedback.

Sources: Rich and Knight, *Artificial Intelligence*, McGraw Hill, 2017.;Russell and Norvig, *Artificial Intelligence: A Modern Approach* (4e), Ch. 1–2 (Introduction, Intelligent Agents).;Poole and Mackworth, *Artificial Intelligence: Foundations of Computational Agents* (2e), selected sections.

# Exit Questions

1. What is PEAS? Create a PEAS description for a *campus navigation app*.
2. Explain partially observable vs fully observable environments with one example each.
3. Why is "rational" not the same as "omniscient"?
4. List and compare any three agent types.